SEO Parameter Implementation

Auditify — On-Page SEO audit: how every parameter is calculated
Source: `Backend/metricServices/seoMetrics.js` | Entry: `seoMetrics(url, $, page)` (line 3689)

1. How each parameter is scored

Every check produces a **fraction between 0 and 1** (common values: 0, 0.5, 0.7, 1). A shared helper `evaluateParameter(score, details, meta)` converts it:

```
score = round(fraction × 100)  •  status = (fraction == 1 → pass) , (fraction ≥ 0.5 → warning) , (fraction < 0.5 → fail)
```

Data sources used: **Cheerio** (parsed HTML DOM), **Puppeteer** `page.evaluate()`, and direct **HTTP fetch** (HEAD/GET). No external PageSpeed/Lighthouse API is used here.

2. Weights & share of the SEO score

23 parameters are weighted into the SEO percentage. Their weights sum to **1.06** (`totalWeight`); the score is divided by that sum, so the “Share” column is each parameter’s true contribution.

Parameter (key)	Weight	Share	Source	Status
`H1`	0.10	9.43%	Cheerio	Full
`Content_Relevance`	0.10	9.43%	Cheerio	Full*
`Image`	0.08	7.55%	Cheerio + HTTP HEAD	Full
`Canonical`	0.08	7.55%	Cheerio	Full
`Contextual_Linking`	0.08	7.55%	Cheerio + HTTP GET	Full
`EEAT`	0.08	7.55%	Page fetch + regex	Full
`Structured_Data`	0.06	5.66%	Puppeteer	Full (presence)
`Meta_Description`	0.05	4.72%	Cheerio	Full
`Sitemap`	0.05	4.72%	HTTP fetch	Full
`Title`	0.04	3.77%	Cheerio	Full
`Title_Uniqueness`	0.04	3.77%	Multi-page HTTP	Full
`Title_Keyword_Optimization`	0.04	3.77%	Multi-page HTTP	Full
`Robots_Txt`	0.04	3.77%	HTTP fetch	Full
`Title_Location_Optimization`	0.03	2.83%	Schema/footer/contact	Full
`Meta_Description_Uniqueness`	0.03	2.83%	Multi-page HTTP	Full
`Heading_Hierarchy`	0.03	2.83%	Cheerio	Full
`URL_Slugs`	0.03	2.83%	URL parsing	Full
`Links`	0.03	2.83%	Cheerio	Full
`Open_Graph`	0.02	1.89%	Cheerio	Full
`Twitter_Card`	0.02	1.89%	Cheerio	Full
`Semantic_Tags`	0.01	0.94%	Cheerio	Full
`Video`	0.01	0.94%	Cheerio	Full
`Social_Links`	0.01	0.94%	Cheerio	Full
TOTAL (weighted)	**1.06**	**100%**		

* Content_Relevance is weighted using its raw `.percentage`, not the standard `.score` (see Section 5).

Display-only (computed and returned, but NOT in the percentage): `URL_Structure`, `Service_Content_Quality`, `Content_Depth_Quality`, `Local_SEO`.

Dead weight: `Duplicate_Content: 0.02` exists in the weights object but is never computed, never in the formula, and never returned.

3. Per-parameter implementation flow

Each parameter below shows the exact decision flow the code follows, in order, with the score it assigns at each branch.

3.1. Title — `Title`

Weight 0.04 • Cheerio `$("title")` • checkTitle()

Does a <title> tag exist? If yes, measure its text length and branch:

- <title> tag missing → **FAIL (0.0)**
- Tag exists but text length == 0 (empty) → **FAIL (0.0)**
- Length < 30 characters (too short) → **WARN (0.5)**
- Length > 60 characters (too long, may truncate in SERP) → **WARN (0.5)**
- Length 30–60 characters (optimal) → **PASS (1.0)**

3.2. Meta Description — `Meta_Description`

Weight 0.05 • Cheerio `meta[name=description]` • checkMetaDescription()

Does the meta description tag exist? If yes, measure the `content` length:

- Tag missing → **FAIL (0.0)**
- Tag exists but content length == 0 → **FAIL (0.0)**
- Length < 50 characters (too short) → **WARN (0.5)**
- Length > 160 characters (too long) → **WARN (0.5)**
- Length 50–160 characters (optimal) → **PASS (1.0)**

3.3. H1 — `H1`

Weight 0.10 • Cheerio `$("h1")` • checkH1()

Count the <h1> tags on the page:

- 0 H1 tags (missing) → **WARN (0.5)** *(note: a warning, not a hard fail)*
- Exactly 1 H1 but its text is empty → **WARN (0.5)**
- Exactly 1 H1 with text → **PASS (1.0)**
- More than 1 H1 tag → **WARN (0.5)**

3.4. Canonical — `Canonical`

Weight 0.08 • Cheerio `link[rel=canonical]` • checkCanonical()

Does a canonical link exist? Then validate it:

- No canonical tag → **WARN (0.5)**
- More than one canonical tag → **FAIL (0.0)**
- Canonical href is empty → **FAIL (0.0)**
- Canonical URL is malformed/unparseable → **FAIL (0.0)**
- Self-referencing (points to this page) → **PASS (1.0)**
- Points to another URL on the **same domain** → **PASS (1.0)**
- Points to an **external domain** → **FAIL (0.0)**

3.5. Image — `Image`

Weight 0.08 • Cheerio `$("img")` + HTTP HEAD (up to 15 unique srcs) • checkImages()

If there are no images → score 1.0. Otherwise a weighted composite is built from four sub-scores:

- **altScore** = images with alt text / total (weight 0.5)
- **meaningfulScore** = images with non-generic alt (≥2 words or >5 chars, not in a blocklist like 'logo/icon/image') / total (weight 0.2)
- **titleScore** = images with a title attribute / total (weight 0.1)
- **sizeScore** = 1 – 0.1 per image >150KB (HTTP HEAD content-length) (weight 0.2)

```
weightedScore = altScore×0.5 + meaningfulScore×0.2 + titleScore×0.1 + sizeScore×0.2
if (score < 0.5) score = 0.5 (floor)  •  if any broken image (bad HTTP/Content-Type) → score forced to 0.5
```

3.6. Video — `Video`

Weight 0.01 • Cheerio `video, iframe[youtube|vimeo]` • checkVideos()

If there are no videos → score 1.0. Otherwise average three sub-scores:

- **embedScore** = 1 (always, embedding is present)
- **lazyScore** = videos with loading='lazy' (or .lazy class) / total
- **metaScore** = videos with itemprop metadata / total

```
score = (embedScore + lazyScore + metaScore) / 3
```

3.7. Heading Hierarchy — `Heading_Hierarchy`

Weight 0.03 • Cheerio `h1..h6` • checkHeadingHierarchy()

Start at score 1, then apply penalties based on heading structure:

- No headings at all on the page → **FAIL (0.0)**
- Multiple H1 tags → **WARN (0.5)**
- Any skipped level (e.g. H2 → H4) or missing H1 → **WARN (0.5)**
- Single H1 and no skipped levels → **PASS (1.0)**

3.8. Semantic Tags — `Semantic_Tags`

Weight 0.01 • Cheerio `main, nav, header, footer, …` • checkSemanticTags()

Checks for HTML5 landmark tags, in order:

- <main> missing OR more than one <main> → **WARN (0.5)**
- Missing any of <header> / <nav> / <footer> → **WARN (0.5)**
- Only <main> present and NO div-class heuristic replacements found → **FAIL (0.0)** (severe, div-only layout)
- Only <main> present but div replacements detected → **WARN (0.5)**
- All major semantic tags properly used → **PASS (1.0)**

3.9. Links — `Links`

Weight 0.03 • Cheerio `$("a")` (no network) • checkLinks()

Counts internal/external/unique links and inspects anchor text. `descRatio = descriptive anchors / total`.

- Any generic anchors ('click here', 'read more', 'here'…) OR descRatio < 0.75 → **WARN (0.5)**
- Otherwise (descriptive anchors, no generic text) → **PASS (1.0)**

3.10. URL Slugs — `URL_Slugs`

Weight 0.03 • URL parsing only • checkSlugs()

Examines the last path segment of the URL:

- Root URL ('/' or empty path) → **PASS (1.0)**
- Penalty triggers (any one → warning): last segment >50 chars; contains uppercase; contains underscores; contains numbers when path depth > 2
- Any penalty triggered → **WARN (0.5)**
- Clean slug (no penalties) → **PASS (1.0)**

3.11. Contextual Linking — `Contextual_Linking`

Weight 0.08 • Cheerio + HTTP GET (up to 150 links) • checkContextualLinks()

Extracts in-content links (vs nav/menu links), computes ratio = contextual / (contextual + menu). Start at score 1, drop to 0.5 if any condition hits:

- Zero contextual links in main content → **WARN (0.5)**

- More than 100 contextual links (spam risk) → **WARN (0.5)**
- Ratio < 0.3 (most links are navigation) → **WARN (0.5)**
- More than 5 important menu pages not linked contextually → **WARN (0.5)**
- Any broken contextual link (HTTP GET fails; links containing 'inventory' are skipped) → **WARN (0.5)**
- None of the above → **PASS (1.0)**

3.12. Content Relevance — `Content_Relevance`

Weight 0.10 • Cheerio body text vs title+meta keywords • checkContentRelevance()

Extracts target keywords from the title + meta description, then matches them against page content (exact, stemmed, 2-word phrases, and ≥5-char brand substrings).

- N = number of target keywords; M = matched keywords
- **P = round(M / N × 100)** — this raw percentage is what feeds the weighted SEO score
- Keyword-stuffing penalty: any word with frequency >7% of body text subtracts 10 → finalScore = clamp(P – penalty, 0, 100)
- Labels (display): finalScore ≥75 = HIGH/pass, ≥40 = MEDIUM/warning, else LOW/fail
- If no keywords could be extracted (N=0) → percentage 0, LOW

```
Weighting note: the score formula multiplies contentRelevanceMetric.percentage (raw P),
NOT the penalized finalScore, and not a 0–1 fraction.
```

3.13. Robots.txt — `Robots_Txt`

Weight 0.04 • HTTP fetch `/robots.txt` (Puppeteer-first, node-fetch fallback) • checkRobotsTxt()

Fetch /robots.txt, then evaluate the `User-agent: *` block:

- File missing (HTTP ≥ 400 and no content) → **WARN (0.5)**
- File present but empty → **WARN (0.5)**
- Full-site block — `Disallow: /` → **FAIL (0.0)**
- Query params blocked — `Disallow: /*?` → **WARN (0.7)**
- Otherwise (valid, not fully blocked) → **PASS (1.0)**

3.14. Sitemap — `Sitemap`

Weight 0.05 • HTTP fetch (robots-declared + /sitemap.xml + /sitemap_index.xml) • checkSitemap()

Try each candidate sitemap URL until one loads, then validate:

- No sitemap found anywhere → **WARN (0.5)**
- Found but invalid structure (no <urlset> / <sitemapindex>) → **FAIL (0.0)**
- Valid but no <lastmod> at all, OR any lastmod older than 180 days → **WARN (0.5)** (outdated)
- Valid and fresh → **PASS (1.0)**

3.15. Structured Data — `Structured_Data`

Weight 0.06 • Puppeteer `script[type=application/ld+json]` • checkStructuredData()

Parses all JSON-LD blocks (presence check only — it does NOT validate schema correctness):

- At least one valid JSON-LD block present → **PASS (1.0)**
- No JSON-LD found → **WARN (0.5)**
- Parse/page error → **FAIL (0.0)**

3.16. Open Graph — `Open_Graph`

Weight 0.02 • Cheerio `meta[property=og:*]` • checkSocial()

Checks required Open Graph tags: `og:title, og:image, og:url`.

- All three required OG tags present → **PASS (1.0)**
- Any required OG tag missing → **WARN (0.5)**

3.17. Twitter Card — `Twitter_Card`

Weight 0.02 • Cheerio twitter meta tags • checkSocial()

Checks required Twitter Card tags: `twitter:card, twitter:title`.

- Both required Twitter tags present → **PASS (1.0)**
- Either missing → **WARN (0.5)**

3.18. Social Links — `Social_Links`

Weight 0.01 • Cheerio `$("a")` vs social-domain list • checkSocial()

Scans all anchors for links to known social domains (facebook, x/twitter, linkedin, instagram, youtube, …).

- At least one social profile link found → **PASS (1.0)**
- No social links found → **WARN (0.5)**

3.19. Title Uniqueness — `Title_Uniqueness`

Weight 0.04 • Multi-page HTTP (up to 5 internal pages) • checkTitleUniqueness() → tuScoreUniqueness()

Samples up to 5 eligible internal pages (sitemap first, else homepage crawl) and fetches each page's <title> once.

- Sample failed / no eligible pages (`!sample.ok`) → **WARN (0.5)** (inconclusive)
- All sampled titles missing (foundCount == 0) → **FAIL (0.0)**
- Otherwise: **score = uniqueCount / pagesChecked** (distinct case-insensitive titles ÷ total pages sampled; missing values dilute the score)

3.20. Meta Description Uniqueness — `Meta_Description_Uniqueness`

Weight 0.03 • Multi-page HTTP (same 5-page sample) • checkMetaDescriptionUniqueness() → tuScoreUniqueness()

Identical logic to Title Uniqueness, applied to the meta description field:

- Sample failed → **WARN (0.5)** (inconclusive)
- All descriptions missing → **FAIL (0.0)**
- Otherwise: **score = uniqueCount / pagesChecked**

3.21. Title Keyword Optimization — `Title_Keyword_Optimization`

Weight 0.04 • Multi-page HTTP • checkTitleKeywordOptimization()

For each sampled page, derives a target keyword and checks whether the page's <title> contains it. Keyword derivation cascade: (1) URL slug → (2) H1 first 3 tokens → (3) top-2 most frequent content words.

- Sample failed (`!sample.ok`) → **WARN (0.5)** (inconclusive)
- Otherwise: **score = optimizedCount / pagesChecked** (titles containing their derived keyword ÷ pages). Match = keyword token, or its ≥4-char stem, is a substring of the title.

3.22. Title Location Optimization — `Title_Location_Optimization`

Weight 0.03 • Schema → footer → contact page → body • checkTitleLocationOptimization()

Resolves the business city/state using a cascade (first hit wins): (1) JSON-LD schema address, (2) footer text, (3) fetched contact page, (4) whole-page body text. Then checks whether the title mentions that location.

- Location cannot be determined (no city and no state) → **WARN (0.5)** (inconclusive)
- Title mentions the resolved city / state / state-abbreviation → **PASS (1.0)**
- Location resolved but title does NOT mention it → **FAIL (0.0)**

3.23. E-E-A-T — EEAT

Weight 0.08 • Trust-page discovery + fetch (cap 5) + regex • checkEEAT()

Discovers and fetches trust pages (About/Contact/Team/Privacy/Terms), then scores five sub-checks 0–2 each. Final fraction = score10 / 10.

- **About (0–2):** >1 story/mission/history term AND ≥150 words → 2; some signals → 1; no about page → 0
- **Contact (0–2):** count of {phone, email, address, contact form}: ≥3 → 2; ≥1 → 1; else 0
- **Author/Team (0–2):** team page/keywords AND credentials → 2; team only → 1; else 0

- **Experience (0–2):** of {years/established, experience, awards, testimonials, case studies, certifications}: ≥2 → 2; ≥1 → 1; else 0
- **Trust (0–2):** count of {HTTPS, privacy policy, terms, review/trust badge}: ≥3 → 2; ≥1 → 1; else 0

```
score10 = about + contact + author + experience + trust  (0–10)  •  fraction = score10 / 10
```

4. Display-only parameters (returned, not scored)

These are fully computed and returned in the response, but are NOT part of the weighted SEO percentage.

4.1. URL Structure — `URL_Structure`

Display-only • URL parsing • checkURLStructure()

Penalizes: uppercase in path, underscores, query parameters, depth > 3 segments.

- Any issue → **WARN (0.5)** / No issues → **PASS (1.0)**
- Has no entry in the weights object → never affects the score (only `URL_Slugs` is weighted).

4.2. Service Content Quality — `Service_Content_Quality`

Display-only (0–10) • Service-page fetch • checkServiceContentQuality()

Finds the best service page, scores four sub-checks 0–2 each (raw 0–8). Returns 0 if no service page / page fails to load.

- **Description (0–2):** of {H1 + ≥40 words, benefits keywords or ≥3 list items, target-audience text}: ≥2 → 2; 1 → 1; else 0
- **Content length (0–2):** ≥150 words → 2; ≥75 → 1; else 0
- **Booking (0–2):** booking embed/form/CTA → 2; any form → 1; else 0
- **Pre-service info (0–2):** of {process, timeline, pricing, requirements, faq}: ≥2 → 2; 1 → 1; else 0

```
rawScore = sum of 4 (0–8)  •  fraction = rawScore / 8
```

4.3. Content Depth Quality — `Content_Depth_Quality`

Display-only (0–10) • Up to 6 typed pages • checkContentDepthQuality()

Classifies and fetches up to 6 typed pages (SRP/VDP/Service/Trade-In/About/Contact). Each page scored on three 0–2 checks, then per-page scores averaged. No targets → 0.5 (inconclusive).

- **Relevance (0–2):** of 5 signals {brand, location, phone/email, year, dealer name}: ≥3 → 2; ≥1 → 1; else 0 (capped at 1 if type-mandatory mentions are missing)
- **Depth (0–2):** word count vs per-type threshold (e.g. VDP 200, SRP 150, contact 100): ≥threshold → 2; ≥60% → 1; else 0
- **Uniqueness (0–2):** max Jaccard similarity (4-gram fingerprint) vs other pages: <0.30 → 2; 0.30–0.60 → 1; >0.60 → 0

```
per-page raw = relevance + depth + uniqueness (0–6) → score10  •  final = mean of
per-page score10, ÷10
```

4.4. Local SEO — `Local_SEO`

Display-only (0–100, avg of 8 sub-signals) • checkLocalSEO()

Aggregates eight local-SEO sub-signals; final score = simple average of the 8 sub-scores. Two sub-signals are marked `partial` (link-detection only; full data needs the Google Places API).

- **NAP_Consistency** = points / 4 (schema name, phone, address with city/state, consistency)
- **LocalBusiness_Schema** = present / 6 (name, address, telephone, geo, openingHours, sameAs); 0 if no LocalBusiness node
- **Location_Targeting** = hits / 3 (city/state in Title, Meta, Body); 0 if no location
- **Local_Keyword_Usage** = cityHits≥2 or ‘near me’ → 1; cityHits==1 → 0.5; else 0
- **Local_Landing_Pages** = dedicated location page → 1; business location only → 0.5; else 0
- **Location_Page_Completeness** = present / 4 (map, hours, directions, phone/NAP)
- **Google_Business_Profile** = GBP/Maps link present → 1; else 0 *(partial)*
- **Review_Signals** = rating schema or review links → 1; star widget → 0.5; else 0 *(partial)*

```
Local_SEO = average(8 sub-signal scores)
```

5. Basis — exactly what each check fetches / inspects

For transparency, this section lists the precise data each parameter reads — the HTML elements, schema fields, HTTP resources, regex patterns and keyword lists it inspects to decide a score.

Title / Meta Description / H1 / Canonical (on-page tags)

- **Title**: text of `$("title")` — existence + character length.
- **Meta Description**: `meta[name="description"]` → `content` attribute length.
- **H1**: count and text of all `$("h1")` elements.
- **Canonical**: `link[rel="canonical"]` → `href`; compared host+path+query against the current URL to classify self / internal / external.

Image

- All `$("img")`: reads `src`, `alt`, `title` attributes.
- 'Meaningful' alt = not in a blocklist (image, logo, icon, pic, photo, spacer, img, …) and ≥2 words or >5 chars.
- Up to 15 unique srcs are fetched via **HTTP HEAD** — reads `content-type` (broken if not `image/*`) and `content-length` (>150KB = heavy).

Video

- `video` tags and `iframe[src*='youtube'|'vimeo']`.
- Reads `loading="lazy"` (or `.lazy` class) and `itemprop` metadata on each.

Heading Hierarchy / Semantic Tags

- **Hierarchy**: all `h1..h6` in document order — checks single H1 and no skipped levels (e.g. H2→H4).
- **Semantic**: presence/count of `main, nav, article, section, header, footer, aside`; heuristic fallback looks for `div[class*="<tag>"]` as a 'potential replacement'.

Links / Contextual Linking

- **Links**: all `$("a")` — internal vs external by hostname; generic-anchor blocklist ('click here', 'read more', 'here', 'learn more', …).
- **Contextual**: links inside `main, article, .content, #content, .post` vs links inside `nav, header, footer, .menu, .sidebar`; relevance judged by an intent/slug/TLD map (e.g. 'sign in'→/login).
- Broken-link check: up to 150 in-content links fetched via **HTTP GET** (URLs containing 'inventory' are skipped).

Content Relevance

- Visible body text (scripts/styles/hidden elements removed) vs keywords extracted from title + meta description.
- Matching is exact word, stemmed word, 2-word phrases, and ≥5-char brand substrings.
- Stuffing penalty uses body-only word frequencies (excludes header/nav/footer); any word >7% frequency = penalty.

URL Slugs / URL Structure

- Parsed from `new URL(url)` — no network.
- **Slugs**: last path segment — length, uppercase, underscores, numbers (when depth > 2).
- **Structure**: full path — uppercase, underscores, query string, segment depth > 3.

Robots.txt / Sitemap / Structured Data (fetched resources)

- **Robots**: **HTTP fetch** of `/robots.txt`; parses the `User-agent: *` block for `Disallow: /` (full block) and `Disallow: /*?` (params).
- **Sitemap**: sitemap URLs from robots.txt + `/sitemap.xml` + `/sitemap_index.xml`; validates `<urlset>`/`<sitemapindex>` and `<lastmod>` freshness (180-day cutoff).
- **Structured Data**: Puppeteer reads all `script[type="application/ld+json"]`, JSON-parses them, and lists `@type` values (presence only).

Open Graph / Twitter Card / Social Links

- **OG**: `meta[property="og:title"|"og:image"|"og:url"]` (required set).
- **Twitter**: `meta[name|property="twitter:card"|"twitter:title"]` (required set).
- **Social Links**: anchors whose hostname matches facebook/x/twitter/linkedin/instagram/youtube/pinterest/tiktok/reddit/medium/…

Title Uniqueness / Meta Uniqueness / Keyword / Location (multi-page)

- Samples up to 5 internal pages (sitemap first, else homepage crawl); excludes inventory/VDP/legal pages and asset URLs.
- **Uniqueness**: each page's `<title>` / meta description, compared case-insensitively for distinct values.
- **Keyword**: target keyword derived from URL slug → H1 → top content words, then matched (substring or ≥4-char stem) against the title.
- **Location**: city/state resolved via cascade — JSON-LD address → footer text → fetched contact page → body text; then checked against the title.

E-E-A-T

- Fetches About/Contact/Team/Privacy/Terms pages (cap 5) and scans text with keyword regex.
- **About**: 'our story / history / founded / established / since YEAR / mission / vision / values / who we are' + word count ≥150.
- **Contact**: phone regex, email regex, street/address, and a contact `<form>`.
- **Author/Team**: team page or team keywords + credential keywords.
- **Experience**: years/established, 'years of experience', awards, testimonials/reviews, case studies, certifications.
- **Trust**: HTTPS protocol, Privacy Policy, Terms & Conditions, review/trust-badge match.

Service Content Quality / Content Depth Quality

- **Service**: best service page — H1 + word count, benefits keywords / ≥3 list items, audience text, booking embed/form/CTA, and pre-service sections (process, timeline, pricing, requirements, faq).
- **Depth**: up to 6 typed pages — relevance signals (brand, location, phone/email, year, dealer name), word count vs per-type threshold (VDP 200, SRP 150, contact 100, …), and uniqueness via Jaccard similarity on 4-gram fingerprints.

Local SEO — the 8 sub-signals (this is the basis you asked about)

- **LocalBusiness_Schema**: from the JSON-LD markup, aggregated across all LocalBusiness + Organization nodes, it fetches `name/legalName`, `address`, `telephone/phone`, `geo/latitude/hasMap`, `openingHours/openingHoursSpecification`, and `sameAs` — score = present fields / 6.
- **NAP_Consistency**: phone from `a[href^="tel:"]` + footer phone regex `(\d{3})\d{3}\d{4}`; name + telephone + address from schema; checks the schema phone matches the on-page phone.
- **Location_Targeting**: resolved city/state checked in 3 zones — `<title>`, meta description, and visible `<body>` text.
- **Local_Keyword_Usage**: scans h1–h3 + title + body for 'near me / nearby / in my area / service area / serving / directions / local' and counts city mentions.
- **Local_Landing_Pages**: homepage links whose path matches `/locations | /store-locator | /stores | /branches | /dealers | /service-areas | /near-me`.
- **Location_Page_Completeness**: on the location/contact page — Google Maps embed/iframe/.map class, opening-hours / day-abbreviation / am-pm patterns, 'get directions' / maps/dir, and a `tel:` or phone number.
- **Google_Business_Profile** *(partial)*: anchors or schema `sameAs` matching `g.page | business.google.com | maps/place | maps.app.goo.gl`.
- **Review_Signals** *(partial)*: `AggregateRating` / `ratingValue` / `Review` schema, links to Yelp/Trustpilot/DealerRater/BBB/cars.com/Google reviews, and star-rating widgets (`[class*="rating"], [itemprop="ratingValue"]`).

6. Final SEO percentage & key notes

The 23 weighted parameter scores (0–100) are multiplied by their weights, summed, then normalized by the total weight (1.06):

```
weightedScore = Σ ( parameterScore × weight )   [Content_Relevance uses .percentage]
totalWeight = 1.06  (sum of the 23 used weights; Duplicate_Content & URL_Structure
excluded)
SEO Percentage = round( weightedScore / totalWeight )
```

Worked example: if all weighted parameters scored 100 → weightedScore = 106 → 106 / 1.06 = **100%**. If H1 and Content_Relevance (the two heaviest, 0.10 each) were 0 and the rest 100 → (106 – 20) / 1.06 = **~81%**.

Things to be aware of

- **Duplicate_Content (0.02)** is a dead weight — in the weights object but never computed, never in the formula, never returned, and excluded from totalWeight.
- **URL_Structure** is computed and returned but never weighted (no entry in weights).
- **Content_Relevance** is the only metric not wrapped by evaluateParameter — it is weighted via raw .percentage (before the stuffing penalty); the penalized finalScore is discarded.
- **Three 0–10 advanced metrics** (Service_Content_Quality, Content_Depth_Quality, Local_SEO) are display-only; among the advanced metrics, only **EEAT** is weighted.
- **Two Local SEO sub-signals** (Google_Business_Profile, Review_Signals) are explicitly partial — link-detection only; full accuracy needs the Google Places API.
- **Structured_Data** is presence-only — it does not validate schema correctness.
- The in-code comment says weights “sum to 1.0”, but the actual totalWeight is **1.06**. Harmless (the code divides by that exact sum), but the comment is inaccurate.

Generated from Backend/metricServices/seoMetrics.js — Auditify On-Page SEO audit.